

ASSIGNMENT

STUDENT INFORMATION SYSTEM

TASK -1 (Database Creation and ER Diagram)

DATABASE CREATION:

```
CREATE DATABASE SISDB;
```

```
USE SISDB;
```

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    date_of_birth DATE,  
    email VARCHAR(100) UNIQUE,  
    phone_number VARCHAR(15)  
);
```

Table: **students**

Columns:

student_id	int AI PK
first_name	varchar(50)
last_name	varchar(50)
date_of_birth	date
email	varchar(100)
phone_number	varchar(15)

```
CREATE TABLE Teacher (  
    teacher_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    email VARCHAR(100) UNIQUE  
);
```

Table: **teacher**

Columns:

teacher_id	int AI PK
first_name	varchar(50)
last_name	varchar(50)
email	varchar(100)

```
CREATE TABLE Courses (  
    course_id VARCHAR(10) PRIMARY KEY,  
    course_name VARCHAR(100),  
    credits INT,  
    teacher_id INT,  
    FOREIGN KEY (teacher_id) REFERENCES Teacher(teacher_id)  
);
```

Table: **courses**

Columns:

course_id	varchar(10) PK
course_name	varchar(100)
credits	int
teacher_id	int

```

CREATE TABLE Enrollments (
    enrollment_id VARCHAR(10) PRIMARY KEY,
    student_id INT,
    course_id VARCHAR(10),
    enrollment_date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id) ,
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

```

Table: enrollments

Columns:

enrollment_id	varchar(10) PK
student_id	int
course_id	varchar(10)
enrollment_date	date

```

CREATE TABLE Payments (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT,
    amount DECIMAL(10,2),
    payment_date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id)
);

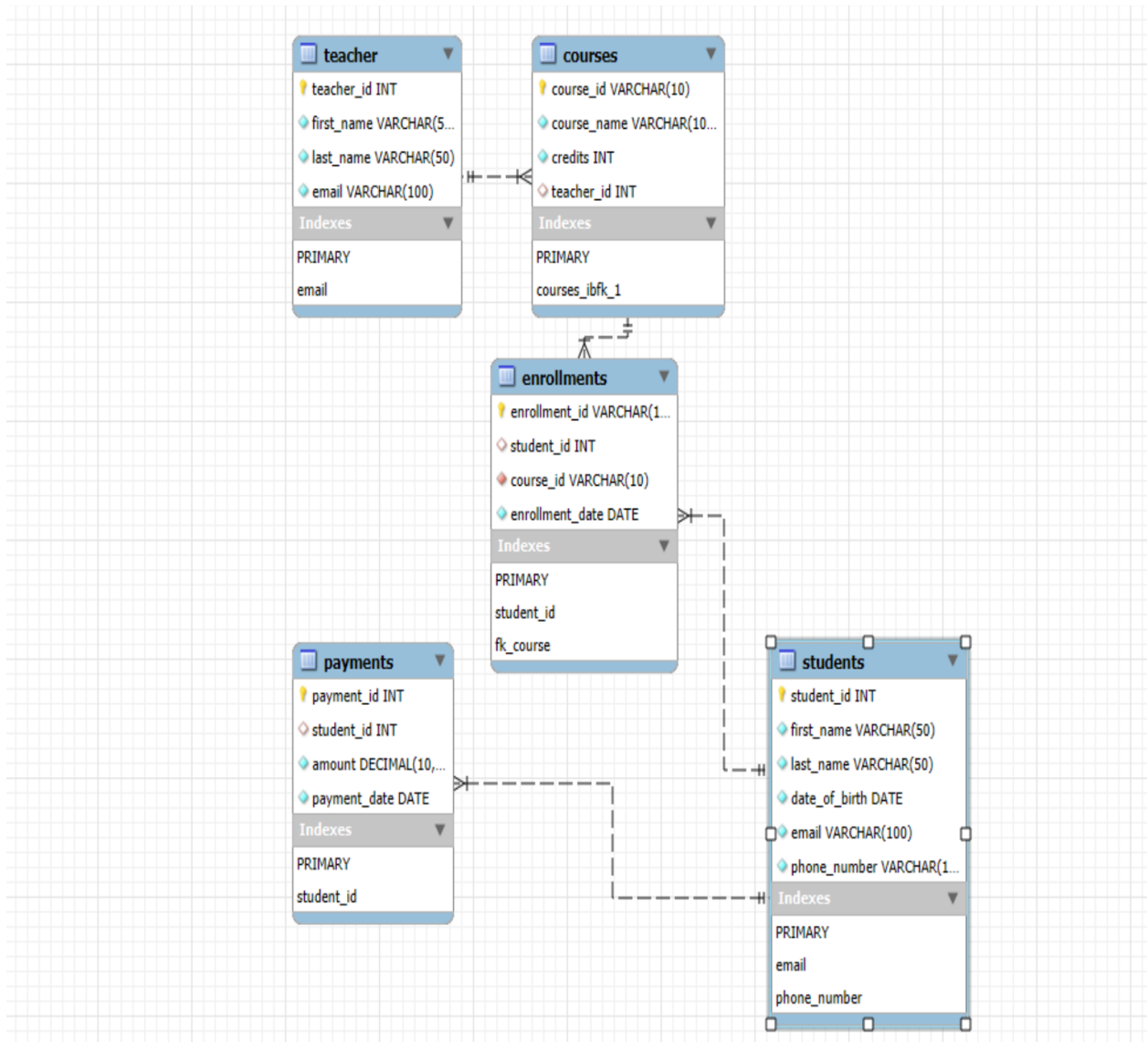
```

Table: payments

Columns:

payment_id	int AI PK
student_id	int
amount	decimal(10,2)
payment_date	date

ER DIAGRAM:



```
INSERT INTO Students (first_name, last_name, date_of_birth, email, phone_number)
VALUES
```

```
('Krish', 'Sharvan', '2000-05-15', 'krish.sharvan@gmail.com', '9987675600'),
('Jane', 'Smith', '2001-07-21', 'jane.smith@gmail.com', '0987654321'),
('Alice', 'Johnson', '2002-03-12', 'alice.johnson@gmail.com', '1122334455'),
('Bob', 'Williams', '2000-09-23', 'bob.williams@gmail.com', '2233445566'),
('Charlie', 'Brown', '1999-12-10', 'charlie.brown@gmail.com', '3344556677'),
('David', 'White', '2003-02-28', 'david.white@gmail.com', '4455667788'),
('Emma', 'Harris', '2001-11-19', 'emma.harris@gmail.com', '5566778899'),
('Frank', 'Martin', '2000-06-30', 'frank.martin@gmail.com', '6677889900'),
('Grace', 'Lee', '2002-08-25', 'grace.lee@gmail.com', '7788990011'),
('Henry', 'Clark', '2001-04-14', 'henry.clark@gmail.com', '8899001122');
```

	student_id	first_name	last_name	date_of_birth	email	phone_number
▶	1	Krish	Sharvan	2000-05-15	krish.sharvan@gmail.com	9987675600
	2	Jane	Smith	2001-07-21	jane.smith@gmail.com	0987654321
	3	Alice	Johnson	2002-03-12	alice.johnson@gmail.com	1122334455
	4	Bob	Williams	2000-09-23	bob.williams@gmail.com	2233445566
	5	Charlie	Brown	1999-12-10	charlie.brown@gmail.com	3344556677
	6	David	White	2003-02-28	david.white@gmail.com	4455667788
	7	Emma	Harris	2001-11-19	emma.harris@gmail.com	5566778899
	8	Frank	Martin	2000-06-30	frank.martin@gmail.com	6677889900
	9	Grace	Lee	2002-08-25	grace.lee@gmail.com	7788990011
	10	Henry	Clark	2001-04-14	henry.clark@gmail.com	8899001122

```
INSERT INTO Payments (student_id, amount, payment_date) VALUES
```

```
(1, 1000.00, '2025-03-25'),
(2, 1200.00, '2025-03-26'),
(3, 1500.00, '2025-03-27'),
(4, 1300.00, '2025-03-28'),
(5, 1100.00, '2025-03-29'),
(6, 1400.00, '2025-03-30'),
(7, 1250.00, '2025-04-01'),
(8, 1350.00, '2025-04-02'),
```

(9, 1450.00, '2025-04-03'),
(10, 1600.00, '2025-04-04');

	payment_id	student_id	amount	payment_date
▶	1	1	1000.00	2025-03-25
	2	2	1200.00	2025-03-26
	3	3	1500.00	2025-03-27
	4	4	1300.00	2025-03-28
	5	5	1100.00	2025-03-29
	6	6	1400.00	2025-03-30
	7	7	1250.00	2025-04-01
	8	8	1350.00	2025-04-02
	9	9	1450.00	2025-04-03
	10	10	1600.00	2025-04-04
•	NULL	NULL	NULL	NULL

INSERT INTO Enrollments (enrollmeny_id,student_id, course_id, enrollment_date)
VALUES

('E1', 'C1', '2025-03-15'),
('E2',' C2', '2025-03-16'),
('E3', 'C3', '2025-03-17'),
('E4', 'C4', '2025-03-18'),
('E5', 'C5', '2025-03-19'),
('E6', 'C6', '2025-03-20'),
('E7', 'C7', '2025-03-21'),
('E8','C8', '2025-03-22'),
('E9', 'C9', '2025-03-23'),
('E10', 'C10', '2025-03-24');

	enrollment_id	student_id	course_id	enrollment_date
▶	E1	1	C11	2025-03-15
	E2	2	C12	2025-03-16
	E3	3	C13	2025-03-17
	E4	4	C14	2025-03-18
	E5	5	C15	2025-03-19
	E6	6	C16	2025-03-20
	E7	7	C17	2025-03-21
	E8	8	C18	2025-03-22
	E9	9	C19	2025-03-23
	E10	10	C20	2025-03-24

INSERT INTO Courses (course_id,course_name, credits, teacher_id) VALUES

('C11','Mathematics', 4, 1),

('C12','Computer Science', 3, 2),

('C13','Physics', 4, 3),

('C14','Chemistry', 4, 4),

('C15','Biology', 3, 5),

('C16','Data Structures', 3, 2),

('C17','Machine Learning', 3, 1),

('C18','Software Engineering', 3, 2),

('C19','Artificial Intelligence', 3, 3),

('C20','Cloud Computing', 3, 4);

	course_id	course_name	credits	teacher_id
▶	C11	Mathematics	4	1
	C12	Computer Science	3	2
	C13	Physics	4	3
	C14	Chemistry	4	4
	C15	Biology	3	5
	C16	Data Structures	3	2
	C17	Machine Learning	3	1
	C18	Software Engineering	3	2
	C19	Artificial Intelligence	3	3
	C20	Cloud Computing	3	4
•	NULL	NULL	NULL	NULL

INSERT INTO Teacher (first_name, last_name, email) VALUES

('Robert', 'Brown', 'robert.brown@gmail.com'),

('siva', 'karthick ', 'siva.karthick@gmail.com'),

('Michael', 'Miller', 'michael.miller@gmail.com'),

('Sarah', 'Wilson', 'sarah.wilson@gmail.com'),

('Daniel', 'Moore', 'daneil.moore@gmail.com'),

('Tamil', 'arasi', 'tamil.arasi@gmail.com'),

('Padma', 'priya', 'padma.priya@gmail.com'),

('sankar', 'velappan', 'sankar.velappan@gmail.com'),

('Michael', 'raj', 'michael.raj@gmail.com'),

('sudha', 'chandran', 'sudha.chandran@gmail.com');

	teacher_id	first_name	last_name	email
▶	1	Robert	Brown	robert.brown@gmail.com
	2	Siva	karthick	siva.karthick@gmail.com
	3	Michael	Miller	michael.miller@gmail.com
	4	Sarah	Wilson	sarah.wilson@gmail.com
	5	Daniel	Moore	daniel.mooren@gmail.com
	6	Tamil	arasi	tamil.arasi@gmail.com
	7	padma	priya	padma.priya@gmail.com
	8	sankar	velappan	sankar.velappan@gmail.com
	9	Michael	raj	michael.raj@gmail.com
	10	sudha	chandran	sudha.chandran@gmail.com
*	NULL	NULL	NULL	NULL

TASK -2 (Select, Where, Between, AND, LIKE)

1. insert into Students (first_name, last_name, date_of_birth, email, phone_number)
values ('John', 'Doe', '1995-08-15', 'john.doe@example.com', '1234567890');

	student_id	first_name	last_name	date_of_birth	email	phone_number
▶	1	Krish	sharvan	2000-05-15	krish.sharvan@gmail.com	9987675600
	2	Jane	Smith	2001-07-21	jane.smith@gmail.com	1122334455
	3	Alice	Johnson	2002-03-12	alice.johnson@gmail.com	1122334455
	4	Bob	Williams	2000-09-23	bob.williams@gmail.com	2233445566
	5	Charlie	Brown	1999-12-10	charlie.brown@gmail.com	3344556677
	6	David	White	2003-02-28	david.white@gmail.com	4455667788
	7	Emma	Harris	2001-11-19	emma.harris@gmail.com	5566778899
	8	Frank	Martin	2000-06-30	frank.martin@gmail.com	6677889900
	9	Grace	Lee	2002-08-25	grace.lee@gmail.com	7788990011
	10	Henry	Clark	2001-04-14	henry.clark@gmail.com	8899001122
	11	John	Doe	1995-08-15	john.doe@example.com	1234567890
•	NULL	NULL	NULL	NULL	NULL	NULL

2. insert into Enrollments (enrollment_id, student_id, course_id, enrollment_date)
values ('E11', 11, 'C14', '2025-03-30');

	enrollment_id	student_id	course_id	enrollment_date
▶	E1	1	C11	2025-03-30
	E2	2	C12	2025-03-16
	E3	3	C13	2025-03-17
	E4	4	C14	2025-03-18
	E5	5	C15	2025-03-19
	E6	6	C16	2025-03-20
	E7	7	C17	2025-03-21
	E8	8	C18	2025-03-22
	E9	9	C19	2025-03-23
	E10	10	C20	2025-03-24
	E11	11	C14	2025-03-30

3. UPDATE Teacher SET email = 'raj@gmail.com' WHERE teacher_id = 9;

	teacher_id	first_name	last_name	email
▶	1	Robert	Brown	robert.brown@gmail.com
	2	Siva	karthick	siva.karthick@gmail.com
	3	Michael	Miller	michael.miller@gmail.com
	4	Sarah	Wilson	sarah.wilson@gmail.com
	5	Daniel	Moore	daniel.mooren@gmail.com
	6	Tamil	arasi	tamil.arasi@gmail.com
	7	padma	priya	padma.priya@gmail.com
	8	sankar	velappan	sankar.velappan@gmail.com
	9	Michael	raj	raj@gmail.com
	10	sudha	chandran	sudha.chandran@gmail.com
•	NULL	NULL	NULL	NULL

4. DELETE FROM enrollments WHERE student_id = 3 AND course_id = 'C13';

	enrollment_id	student_id	course_id	enrollment_date
▶	E1	1	C11	2025-03-30
	E2	2	C12	2025-03-16
	E4	4	C14	2025-03-18
	E5	5	C15	2025-03-19
	E6	6	C16	2025-03-20
	E7	7	C17	2025-03-21
	E8	8	C18	2025-03-22
	E9	9	C19	2025-03-23
	E10	10	C20	2025-03-24
	E11	11	C14	2025-03-30

5. UPDATE Courses SET teacher_id = 6 WHERE course_id = 'C11';

	course_id	course_name	credits	teacher_id
▶	C11	Mathematics	4	6
	C12	Computer Science	3	2
	C13	Physics	4	3
	C14	Chemistry	4	4
	C15	Biology	3	5
	C16	Data Structures	3	2
	C17	Machine Learning	3	1
	C18	Software Engineering	3	2
	C19	Artificial Intelligence	3	3
	C20	Cloud Computing	3	4
•	NULL	NULL	NULL	NULL

6. DELETE from enrollments where student_id=8;
DELETE from students where student_id=8;

	enrollment_id	student_id	course_id	enrollment_date
▶	E1	1	C11	2025-03-30
	E2	2	C12	2025-03-16
	E4	4	C14	2025-03-18
	E5	5	C15	2025-03-19
	E6	6	C16	2025-03-20
	E7	7	C17	2025-03-21
	E9	9	C19	2025-03-23
	E10	10	C20	2025-03-24
	E11	11	C14	2025-03-30

	student_id	first_name	last_name	date_of_birth	email	phone_number
▶	1	Krish	sharvan	2000-05-15	krish.sharvan@gmail.com	9987675600
	2	Jane	Smith	2001-07-21	jane.smith@gmail.com	0987654321
	3	Alice	Johnson	2002-03-12	alice.johnson@gmail.com	1122334455
	4	Bob	Williams	2000-09-23	bob.williams@gmail.com	2233445566
	5	Charlie	Brown	1999-12-10	charlie.brown@gmail.com	3344556677
	6	David	White	2003-02-28	david.white@gmail.com	4455667788
	7	Emma	Harris	2001-11-19	emma.harris@gmail.com	5566778899
	9	Grace	Lee	2002-08-25	grace.lee@gmail.com	7788990011
	10	Henry	Clark	2001-04-14	henry.clark@gmail.com	8899001122
	11	John	Doe	1995-08-15	john.doe@example.com	1234567890
*	NULL	NULL	NULL	NULL	NULL	NULL

7. UPDATE payments SET amount = 6000 WHERE student_id = 2;

	payment_id	student_id	amount	payment_date
▶	1	1	1000.00	2025-03-25
	2	2	6000.00	2025-03-26
	3	3	1500.00	2025-03-27
	4	4	1300.00	2025-03-28
	5	5	1100.00	2025-03-29
	6	6	1400.00	2025-03-30
	7	7	1250.00	2025-04-01
	8	8	1350.00	2025-04-02
	9	9	1450.00	2025-04-03
	10	10	1600.00	2025-04-04
*	NULL	NULL	NULL	NULL

TASK -3 (Aggregate functions, Having, Order By, Group By and Joins)

1. select s.student_id, s.first_name, s.last_name, sum(p.amount) as Total_Amount from Students s join Payments p on s.student_id = p.student_id where s.student_id = 6;

	student_id	first_name	last_name	TOTAL_AMOUNT
▶	6	David	White	1400.00

2. select c.course_name,count(e.student_id) as Total_count from enrollments e join courses c on c.course_id=e.course_id group by c.course_name;

	course_name	Total_count
▶	Mathematics	1
	Computer Science	1
	Chemistry	2
	Biology	1
	Data Structures	1
	Machine Learning	1
	Artificial Intelligence	1
	Cloud Computing	1

3. Select s.first_name,s.last_name from enrollments e left join students s on s.student_id=e.student_id;

	first_name	last_name
▶	Krish	sharvan
	Jane	Smith
	Bob	Williams
	Charlie	Brown
	David	White
	Emma	Harris
	Grace	Lee
	Henry	Clark
	John	Doe

4. `select s.first_name, s.last_name, c.course_name from Students s join Enrollments e on s.student_id = e.student_id join Courses c on e.course_id = c.course_id;`

	first_name	last_name	course_name
▶	Krish	sharvan	Mathematics
	Jane	Smith	Computer Science
	Bob	Williams	Chemistry
	Charlie	Brown	Biology
	David	White	Data Structures
	Emma	Harris	Machine Learning
	Grace	Lee	Artificial Intelligence
	Henry	Clark	Cloud Computing
	John	Doe	Chemistry

5. `select t.first_name, t.last_name, c.course_name from teacher t join Courses c on t.teacher_id = c.teacher_id;`

	first_name	last_name	course_name
▶	Tamil	arasi	Mathematics
	Siva	karthick	Computer Science
	Michael	Miller	Physics
	Sarah	Wilson	Chemistry
	Daniel	Moore	Biology
	Siva	karthick	Data Structures
	Robert	Brown	Machine Learning
	Siva	karthick	Software Engineering
	Michael	Miller	Artificial Intelligence
	Sarah	Wilson	Cloud Computing

6. `select s.first_name, s.last_name from students s left join payments p ON s.student_id = p.student_id where p.payment_id is null;`

	first_name	last_name
▶	John	Doe

7. `select s.*, e.enrollment_date from Students s join Enrollments e on s.student_id = e.student_id join Courses c on e.course_id = c.course_id where c.course_id = 'C14';`

	student_id	first_name	last_name	date_of_birth	email	phone_number	enrollment_date
▶	4	Bob	Williams	2000-09-23	bob.williams@gmail.com	2233445566	2025-03-18
	11	John	Doe	1995-08-15	john.doe@example.com	1234567890	2025-03-30

8. `select c.course_name from courses c left join enrollments e on c.course_id = e.course_id where e.enrollment_id is null;`

	course_name
▶	Physics
	Software Engineering

9. select e1.student_id from Enrollments e1 join Enrollments e2 on e1.student_id = e2.student_id and e1.course_id != e2.course_id;

	student_id
--	------------

10. select t.* from teacher t left join courses c on c.teacher_id=t.teacher_id where c.course_id is null;

	teacher_id	first_name	last_name	email
▶	7	padma	priya	padma.priya@gmail.com
	8	sankar	velappan	sankar.velappan@gmail.com
	9	Michael	raj	raj@gmail.com
	10	sudha	chandran	sudha.chandran@gmail.com

TASK -4 (Subquery and its type)

1. select floor(avg(student_count)) as Average from (select course_id, count(student_id) as student_count from Enrollments group by course_id) as course_enrollments;

	Average
▶	1

2. select s.* from payments p join students s on s.student_id =p.student_id where p.amount=(select max(amount) from payments);

	student_id	first_name	last_name	date_of_birth	email	phone_number
▶	2	Jane	Smith	2001-07-21	jane.smith@gmail.com	0987654321

3. select e.course_id, count(e.student_id) as Maximum_count from Enrollments e join (select course_id, count(student_id) as max_enrollment from Enrollments group by course_id order by max_enrollment desc limit 1) as max_course on e.course_id = max_course.course_id group by e.course_id;

	course_id	Maximum_count
▶	C14	2

4. select t.teacher_id, t.first_name, t.last_name, (select sum(p.amount) from Payments p join Enrollments e on p.student_id = e.student_id where e.course_id in (select course_id from Courses where teacher_id = t.teacher_id)) as total_payments from Teacher t;

	teacher_id	first_name	last_name	total_payments
▶	1	Robert	Brown	1250.00
	2	Siva	karthick	7400.00
	3	Michael	Miller	1450.00
	4	Sarah	Wilson	2900.00
	5	Daniel	Moore	1100.00
	6	Tamil	arasi	1000.00
	7	padma	priya	NULL
	8	sankar	velappan	NULL
	9	Michael	raj	NULL
	10	sudha	chandran	NULL

5. select s.student_id, s.first_name, s.last_name from Students s where (select count(distinct e.course_id) from Enrollments e where e.student_id = s.student_id) = (select count(*) from Courses);

	student_id	first_name	last_name
*	NULL	NULL	NULL

6. select t.teacher_id, t.first_name, t.last_name from Teacher t where not exists (select 1 from Courses c where c.teacher_id = t.teacher_id);

	teacher_id	first_name	last_name
▶	7	padma	priya
	8	sankar	velappan
	9	Michael	raj
	10	sudha	chandran
*	NULL	NULL	NULL

7. select avg(age) as avg_age from (select year(curdate()) - year(date_of_birth) as age from Students) as student_ages;

	avg_age
▶	24.6000

8. select course_id, course_name from Courses where course_id not in (select distinct course_id from Enrollments);

	course_id	course_name
▶	C13	Physics
	C18	Software Engineering
*	NULL	NULL

9. select e.student_id, e.course_id, sum(p.amount) as total_payment from Payments p join Enrollments e on p.student_id = e.student_id group by e.student_id, e.course_id;

	student_id	course_id	total_payment
▶	1	C11	1000.00
	2	C12	6000.00
	4	C14	1300.00
	5	C15	1100.00
	6	C16	1400.00
	7	C17	1250.00
	9	C19	1450.00
	10	C20	1600.00

10. select student_id, count(payment_id) as payment_count from Payments group by student_id having count(payment_id) > 1;

	student_id	payment_count
--	------------	---------------

11. select s.*, sum(p.amount) as Total_payment from Students s join Payments p on s.student_id = p.student_id group by s.student_id, s.first_name, s.last_name;

	student_id	first_name	last_name	date_of_birth	email	phone_number	Total_payment
▶	1	Krish	sharvan	2000-05-15	krish.sharvan@gmail.com	9987675600	1000.00
	2	Jane	Smith	2001-07-21	jane.smith@gmail.com	0987654321	6000.00
	3	Alice	Johnson	2002-03-12	alice.johnson@gmail.com	1122334455	1500.00
	4	Bob	Williams	2000-09-23	bob.williams@gmail.com	2233445566	1300.00
	5	Charlie	Brown	1999-12-10	charlie.brown@gmail.com	3344556677	1100.00
	6	David	White	2003-02-28	david.white@gmail.com	4455667788	1400.00
	7	Emma	Harris	2001-11-19	emma.harris@gmail.com	5566778899	1250.00
	9	Grace	Lee	2002-08-25	grace.lee@gmail.com	7788990011	1450.00
	10	Henry	Clark	2001-04-14	henry.clark@gmail.com	8899001122	1600.00

12. select c.course_name, count(e.student_id) as student_count from Courses c left join Enrollments e on c.course_id = e.course_id group by c.course_name;

	course_name	student_count
▶	Mathematics	1
	Computer Science	1
	Physics	0
	Chemistry	2
	Biology	1
	Data Structures	1
	Machine Learning	1
	Software Engineering	0
	Artificial Intelligence	1
	Cloud Computing	1

13. select round(avg(avg_payment),2) as Average_payments from (select s.student_id, s.first_name, avg(p.amount) as avg_payment from Students s join Payments p on s.student_id = p.student_id group by s.student_id, s.first_name)as student_avg_payments;

	Average_payments
▶	1844.44

JDBC CONNECTIVITY

Task 1: Define Classes

Entity Package:

1. Student Class

```
package entity;
```

```
import java.util.ArrayList;
```

```
import java.util.Date;
```

```
import java.util.List;
```

```
public class Student {
```

```
    private int studentId;
```

```
    private String firstName;
```

```
    private String lastName;
```

```
    private String dateOfBirth;
```

```
    private String email;
```

```
    private String phoneNumber;
```

```
    private List<Enrollment> enrollments;
```

```
    private List<Payment> payments;
```

```
    // Default Constructor
```

```
    public Student() {
```

```
    this.enrollments = new ArrayList<>();  
    this.payments = new ArrayList<>();  
}
```

// Parameterized Constructor

```
public Student(int studentId, String firstName, String lastName, String  
dateOfBirth, String email, String phoneNumber) {  
    this.studentId = studentId;  
    this.firstName = firstName;  
    this.lastName = lastName;  
    this.dateOfBirth = dateOfBirth;  
    this.email = email;  
    this.phoneNumber = phoneNumber;  
    this.enrollments = new ArrayList<>();  
    this.payments = new ArrayList<>();  
}
```

// Getters and Setters

```
public int getStudentId() {  
    return studentId;  
}
```

```
public void setStudentId(int studentId) {  
    this.studentId = studentId;  
}
```

```
public String getFirstName() {  
    return firstName;  
}
```

```
public void setFirstName(String firstName) {  
    this.firstName = firstName;  
}
```

```
public String getLastName() {  
    return lastName;  
}
```

```
public void setLastName(String lastName) {  
    this.lastName = lastName;  
}
```

```
public String getDateOfBirth() {  
    return dateOfBirth;  
}
```

```
public void setDateOfBirth(String dateOfBirth) {  
    this.dateOfBirth = dateOfBirth;  
}
```

```
public String getEmail() {  
    return email;  
}
```

```
}
```

```
public void setEmail(String email) {  
    this.email = email;  
}
```

```
public String getPhoneNumber() {  
    return phoneNumber;  
}
```

```
public void setPhoneNumber(String phoneNumber) {  
    this.phoneNumber = phoneNumber;  
}
```

```
public List<Enrollment> getEnrollments() {  
    return enrollments;  
}
```

```
public void setEnrollments(List<Enrollment> enrollments) {  
    this.enrollments = enrollments;  
}
```

```
public List<Payment> getPayments() {  
    return payments;  
}
```

```
public void setPayments(List<Payment> payments) {  
    this.payments = payments;  
}
```

```
public void enrollInCourse(String enrollmentId, Course course) {  
    Enrollment enrollment = new Enrollment(enrollmentId, this, course,  
new Date());  
    enrollments.add(enrollment);  
}
```

```
public void updateStudentInfo(String firstName, String lastName,  
String dateOfBirth, String email, String phoneNumber) {  
    this.firstName = firstName;  
    this.lastName = lastName;  
    this.dateOfBirth = dateOfBirth;  
    this.email = email;  
    this.phoneNumber = phoneNumber;  
}
```

```
public void makePayment(int paymentId, double amount, Date  
paymentDate) {  
    Payment payment = new Payment(paymentId, this, amount,  
paymentDate); // Using the constructor with paymentId  
    payments.add(payment);  
}
```

```
public void displayStudentInfo() {  
    System.out.println("Student ID: " + studentId);  
    System.out.println("Name: " + firstName + " " + lastName);  
}
```

```

        System.out.println("Date of Birth: " + dateOfBirth);
        System.out.println("Email: " + email);
        System.out.println("Phone Number: " + phoneNumber);
    }

    public List<Course> getEnrolledCourses() {
        List<Course> courses = new ArrayList<>();
        for (Enrollment enrollment : enrollments) {
            courses.add(enrollment.getCourse());
        }
        return courses;
    }

    public List<Payment> getPaymentHistory() {
        return payments;
    }
}

```

@Override

```

public String toString() {
    return "Student{" +
        "studentId=" + studentId +
        ", firstName=" + firstName + "\" +
        ", lastName=" + lastName + "\" +
        ", dateOfBirth=" + dateOfBirth + "\" +
        ", email=" + email + "\" +
        ", phoneNumber=" + phoneNumber + "\" +
        '}'";
}
}

```



```
}
```

2.Course Class

```
package entity;
```

```
import java.util.ArrayList;
```

```
import java.util.Date;
```

```
import java.util.List;
```

```
public class Course {
```

```
    private String courseId;
```

```
    private String courseName;
```

```
    private int credits;
```

```
    private int teacherId; // Matches DB
```

```
    private String courseCode;
```

```
    private Teacher teacher; // Optional - for full object mapping
```

```
    private List<Enrollment> enrollments;
```

```
    public Course() {
```

```
        this.enrollments = new ArrayList<>();
```

```
    }
```

```
    public Course(String courseId, String courseName, int credits, int  
teacherId, String courseCode) {
```

```
        this.courseId = courseId;
```

```
        this.courseName = courseName;
```

```
    this.credits = credits;
    this.teacherId = teacherId;
    this.courseCode = courseCode;
    this.enrollments = new ArrayList<>();
}
```

// Overloaded constructor without teacherId (in case it's not yet assigned)

```
public Course(String courseId, String courseName, int credits, String
courseCode) {
    this(courseId, courseName, credits, 0, courseCode); // teacherId = 0
=> unassigned
}
```

// Getters and setters

```
public String getCourseId() {
    return courseId;
}
```

```
public void setCourseId(String courseId) {
    this.courseId = courseId;
}
```

```
public String getCourseName() {
    return courseName;
}
```

```
public void setCourseName(String courseName) {  
    this.courseName = courseName;  
}
```

```
public int getCredits() {  
    return credits;  
}
```

```
public void setCredits(int credits) {  
    this.credits = credits;  
}
```

```
public int getTeacherId() {  
    return teacherId;  
}
```

```
public void setTeacherId(int teacherId) {  
    this.teacherId = teacherId;  
}
```

```
public String getCourseCode() {  
    return courseCode;  
}
```

```
public void setCourseCode(String courseCode) {  
    this.courseCode = courseCode;  
}
```

```
}
```

```
public Teacher getTeacher() {  
    return teacher;  
}
```

```
public void setTeacher(Teacher teacher) {  
    this.teacher = teacher;  
}
```

```
public List<Enrollment> getEnrollments() {  
    return enrollments;  
}
```

```
public void setEnrollments(List<Enrollment> enrollments) {  
    this.enrollments = enrollments;  
}
```

```
public void assignTeacher(Teacher teacher) {  
    this.teacher = teacher;  
    this.teacherId = teacher.getTeacherId(); // Keep both updated  
}
```

```
public void enrollStudent(String enrollmentId, Student student, Date  
enrollmentDate) {  
    Enrollment enrollment = new Enrollment(enrollmentId, student, this,  
enrollmentDate);
```

```

        enrollments.add(enrollment);
    }

    @Override
    public String toString() {
        return "Course{" +
            "courseId=" + courseId + "\" +
            ", courseName=" + courseName + "\" +
            ", credits=" + credits +
            ", teacherId=" + teacherId +
            ", courseCode=" + courseCode + "\" +
            ", teacher=" + (teacher != null ? teacher.getFirstName() + " " +
teacher.getLastName() : "None") +
            '}';
    }
}

```

3. Teacher Class

```

package entity;

import java.util.ArrayList;
import java.util.List;

public class Teacher {
    private int teacherId;
    private String firstName;
    private String lastName;

```

```
private String email;
```

```
private List<Course> assignedCourses;
```

```
// Default constructor
```

```
public Teacher() {
```

```
    this.assignedCourses = new ArrayList<>();
```

```
}
```

```
// Parameterized constructor
```

```
public Teacher(int teacherId, String firstName, String lastName, String  
email) {
```

```
    this.teacherId = teacherId;
```

```
    this.firstName = firstName;
```

```
    this.lastName = lastName;
```

```
    this.email = email;
```

```
    this.assignedCourses = new ArrayList<>();
```

```
}
```

```
// Getters and Setters
```

```
public int getTeacherId() {
```

```
    return teacherId;
```

```
}
```

```
public void setTeacherId(int teacherId) {
```

```
    this.teacherId = teacherId;
```

```
}
```

```
public String getFirstName() {  
    return firstName;  
}
```

```
public void setFirstName(String firstName) {  
    this.firstName = firstName;  
}
```

```
public String getLastName() {  
    return lastName;  
}
```

```
public void setLastName(String lastName) {  
    this.lastName = lastName;  
}
```

```
public String getEmail() {  
    return email;  
}
```

```
public void setEmail(String email) {  
    this.email = email;  
}
```

```
public List<Course> getAssignedCourses() {  
    return assignedCourses;  
}
```

```
public void setAssignedCourses(List<Course> assignedCourses) {  
    this.assignedCourses = assignedCourses;  
}
```

```
public void assignCourse(Course course) {  
    if (!assignedCourses.contains(course)) {  
        assignedCourses.add(course);  
    }  
}
```

// Method to remove a course from this teacher

```
public void removeCourse(Course course) {  
    assignedCourses.remove(course);  
}
```

@Override

```
public String toString() {  
    return "Teacher{" +  
        "teacherId=" + teacherId +  
        ", firstName=" + firstName + "\" +  
        ", lastName=" + lastName + "\" +  
        ", email=" + email + "\" +
```



```

        ", assignedCourses=" + (assignedCourses != null ?
assignedCourses.size() : 0) +
        "';
    }
}

```

4.Enrollment Class

package entity;

import java.util.Date;

```

public class Enrollment {
    private String enrollmentId;
    private Student student;
    private Course course;
    private Date enrollmentDate;

    // Default constructor
    public Enrollment() {}

    // Parameterized constructor
    public Enrollment(String enrollmentId, Student student, Course course,
Date enrollmentDate) {
        this.enrollmentId = enrollmentId;
        this.student = student;
        this.course = course;
        this.enrollmentDate = enrollmentDate;
    }
}

```

```
}
```

```
// Getters and Setters
```

```
public String getEnrollmentId() {  
    return enrollmentId;  
}
```

```
public void setEnrollmentId(String enrollmentId) {  
    this.enrollmentId = enrollmentId;  
}
```

```
public Student getStudent() {  
    return student;  
}
```

```
public void setStudent(Student student) {  
    this.student = student;  
}
```

```
public Course getCourse() {  
    return course;  
}
```

```
public void setCourse(Course course) {  
    this.course = course;  
}
```

```
public Date getEnrollmentDate() {  
    return enrollmentDate;  
}
```

```
public void setEnrollmentDate(Date enrollmentDate) {  
    this.enrollmentDate = enrollmentDate;  
}
```

@Override

```
public String toString() {  
    return "Enrollment{" +  
        "enrollmentId=" + enrollmentId + "\" +  
        ", student=" + (student != null ? student.getFirstName() + " " +  
student.getLastName() : "Unknown") +  
        ", course=" + (course != null ? course.getCourseName() :  
"Unknown") +  
        ", enrollmentDate=" + enrollmentDate +  
        "}";  
}  
}
```

5.Payment Class

```
package entity;
```

```
import java.util.Date;
```

```
public class Payment {  
    private int paymentId;  
    private Student student;  
    private double amount;  
    private Date paymentDate;  
  
    // Default constructor  
    public Payment() {}  
  
    public Payment(int paymentId, Student student, double amount, Date  
paymentDate) {  
        this.paymentId = paymentId;  
        this.student = student;  
        this.amount = amount;  
        this.paymentDate = paymentDate;  
    }  
  
    // Getters and Setters  
    public int getPaymentId() {  
        return paymentId;  
    }  
  
    public void setPaymentId(int paymentId) {  
        this.paymentId = paymentId;  
    }  
}
```

```
public Student getStudent() {  
    return student;  
}
```

```
public void setStudent(Student student) {  
    this.student = student;  
}
```

```
public double getAmount() {  
    return amount;  
}
```

```
public void setAmount(double amount) {  
    this.amount = amount;  
}
```

```
public Date getPaymentDate() {  
    return paymentDate;  
}
```

```
public void setPaymentDate(Date paymentDate) {  
    this.paymentDate = paymentDate;  
}
```

@Override

```

public String toString() {
    return "Payment{" +
        "paymentId=" + paymentId +
        ", student=" + (student != null ? student.getFirstName() + " " +
student.getLastName() : "Unknown") +
        ", amount=" + amount +
        ", paymentDate=" + paymentDate +
        '}';
}
}

```

Sis Package

1.Sis Class

```

package sis;

import dao.ISisdb;
import dao.ISisdbImpl;
import entity.Course;
import entity.Enrollment;
import entity.Payment;
import entity.Student;
import entity.Teacher;

import java.util.Date;
import java.util.List;
import exception.*;

```

```
public class Sis {

    private ISisdb sisDb;

    public Sis() {
        sisDb = new ISisdbImpl();
    }

    // Add Teacher
    public void addTeacher(Teacher teacher) {
        try {
            sisDb.addTeacher(teacher);
        } catch (InvalidTeacherDataException e) {
            System.err.println("Error adding teacher: " + e.getMessage());
        }
    }

    // Add Course
    public void addCourse(Course course) {
        try {
            sisDb.addCourse(course);
        } catch (InvalidCourseDataException e) {
            System.err.println("Error adding course: " + e.getMessage());
        }
    }
}
```

// Add Student

```
public void addStudent(Student student) {  
    try {  
        sisDb.addStudent(student);  
    } catch (InvalidStudentDataException e) {  
        System.err.println("Error adding student: " + e.getMessage());  
    }  
}
```

// Add Payment

```
public void addPayment(Student student, int paymentId, double amount,  
Date paymentDate) {  
    try {  
        Payment payment = new Payment(paymentId, student, amount,  
paymentDate);  
        sisDb.addPayment(payment);  
    } catch (PaymentValidationException e) {  
        System.err.println("Error adding payment: " + e.getMessage());  
    }  
}
```

// Add Enrollment

```
public void addEnrollment(Student student, Course course, String  
enrollmentId, Date enrollmentDate) {  
    try {  
        Enrollment enrollment = new Enrollment(enrollmentId, student,  
course, enrollmentDate);
```



```
        sisDb.addEnrollment(enrollment);
    } catch (DuplicateEnrollmentException e) {
        System.err.println("Error enrolling student: " + e.getMessage());
    } catch (CourseNotFoundException e) {
        System.err.println("Course not found: " + e.getMessage());
    } catch (StudentNotFoundException e) {
        System.err.println("Student not found: " + e.getMessage());
    }
}
```

// Assign Teacher to a Course

```
public void assignCourseToTeacher(Course course, Teacher teacher) {
    try {
        sisDb.assignCourseToTeacher(course, teacher);
    } catch (CourseNotFoundException e) {
        System.err.println("Course not found: " + e.getMessage());
    } catch (TeacherNotFoundException e) {
        System.err.println("Teacher not found: " + e.getMessage());
    }
}
```

// Get Enrollments for a Student

```
public List<Enrollment> getEnrollmentsForStudent(Student student) {
    try {
        return sisDb.getEnrollmentsForStudent(student.getId());
    } catch (StudentNotFoundException e) {
```

```
        System.err.println("Error retrieving enrollments for student: " +  
e.getMessage());
```

```
        return null; // Return null or an empty list, depending on your  
preference
```

```
    }  
}
```

```
// Get Courses for a Teacher
```

```
public List<Course> getCoursesForTeacher(Teacher teacher) {  
    try {  
        return sisDb.getCoursesForTeacher(teacher.getTeacherId());  
    } catch (TeacherNotFoundException e) {  
        System.err.println("Error retrieving courses for teacher: " +  
e.getMessage());  
        return null; // Return null or an empty list  
    }  
}
```

```
// Retrieve all Students
```

```
public List<Student> getAllStudents() {  
    return sisDb.getAllStudents();  
}
```

```
// Retrieve all Teachers
```

```
public List<Teacher> getAllTeachers() {  
    return sisDb.getAllTeachers();  
}
```

```

// Retrieve all Courses
public List<Course> getAllCourses() {
    return sisDb.getAllCourses();
}

// Retrieve all Payments
public List<Payment> getAllPayments() {
    return sisDb.getAllPayments();
}
}

```

Dao Package

1. Isisdb Interface

```

package dao;

import entity.Course;
import entity.Enrollment;
import entity.Payment;
import entity.Student;
import entity.Teacher;

import java.util.List;
import exception.*;

public interface ISisdb {
    void addStudent(Student student) throws InvalidStudentDataException;
}

```

```
void addTeacher(Teacher teacher) throws InvalidTeacherDataException;  
void addCourse(Course course) throws InvalidCourseDataException;  
void addPayment(Payment payment) throws  
PaymentValidationException;
```

```
void addEnrollment(Enrollment enrollment) throws  
DuplicateEnrollmentException, CourseNotFoundException,  
StudentNotFoundException;
```

```
void assignCourseToTeacher(Course course, Teacher teacher) throws  
CourseNotFoundException, TeacherNotFoundException;
```

```
Student getStudentById(int studentId) throws  
StudentNotFoundException;
```

```
Teacher getTeacherById(int teacherId) throws  
TeacherNotFoundException;
```

```
Course getCourseById(int courseId) throws CourseNotFoundException;
```

```
List<Student> getAllStudents();
```

```
List<Teacher> getAllTeachers();
```

```
List<Course> getAllCourses();
```

```
List<Payment> getAllPayments();
```

```
List<Enrollment> getEnrollmentsForStudent(int studentId) throws  
StudentNotFoundException;
```

```
List<Course> getCoursesForTeacher(int teacherId) throws  
TeacherNotFoundException;
```

```
List<Enrollment> generateEnrollmentReport(String courseName);
```

```
}
```

2.ISisdBImpl Class

```
package dao;
```

```
import entity.Course;
```

```
import entity.Enrollment;
```

```
import entity.Payment;
```

```
import entity.Student;
```

```
import entity.Teacher;
```

```
import util.DBConnUtil;
```

```
import java.sql.*;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class ISisdBImpl implements ISisdB {
```

```
    @Override
```

```
    public void addStudent(Student student) {
```

```
        String query = "INSERT INTO students (student_id, first_name,  
last_name, date_of_birth, email, phone_number) VALUES (?, ?, ?, ?, ?,  
?)";
```

```
        try (Connection con = DBConnUtil.getDbConnection();
```

```
            PreparedStatement ps = con.prepareStatement(query)) {
```

```
            ps.setInt(1, student.getStudentId());
```

```

        ps.setString(2, student.getFirstName());
        ps.setString(3, student.getLastName());
        ps.setString(4, student.getDateOfBirth());
        ps.setString(5, student.getEmail());
        ps.setString(6, student.getPhoneNumber());

        ps.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

@Override

```

public void addTeacher(Teacher teacher) {
    String query = "INSERT INTO teacher (teacher_id, first_name,
last_name, email) VALUES (?, ?, ?, ?)";

```

```

    try (Connection con = DBConnUtil.getDbConnection();
        PreparedStatement ps = con.prepareStatement(query)) {

```

```

        ps.setInt(1, teacher.getTeacherId());
        ps.setString(2, teacher.getFirstName());
        ps.setString(3, teacher.getLastName());
        ps.setString(4, teacher.getEmail());

```

```

        ps.executeUpdate();

```

```
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

@Override

```
public void addCourse(Course course) {  
    String query = "INSERT INTO courses (course_id, course_name,  
credits, teacher_id, course_code) VALUES (?, ?, ?, ?, ?)";  
  
    try (Connection con = DBConnUtil.getDbConnection();  
        PreparedStatement ps = con.prepareStatement(query)) {  
  
        ps.setString(1, course.getCourseId());  
        ps.setString(2, course.getCourseName());  
        ps.setInt(3, course.getCredits());  
        ps.setInt(4, course.getTeacherId());  
        ps.setString(5, course.getCourseCode());  
  
        ps.executeUpdate();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

@Override

```
public void addEnrollment(Enrollment enrollment) {  
    String query = "INSERT INTO enrollments (enrollment_id,  
student_id, course_id, enrollment_date) VALUES (?, ?, ?, ?)";  
  
    try (Connection con = DBConnUtil.getDbConnection();  
        PreparedStatement ps = con.prepareStatement(query)) {  
  
        ps.setString(1, enrollment.getEnrollmentId());  
        ps.setInt(2, enrollment.getStudent().getStudentId());  
        ps.setString(3, enrollment.getCourse().getCourseId());  
        ps.setDate(4, new  
java.sql.Date(enrollment.getEnrollmentDate().getTime()));  
  
        ps.executeUpdate();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

@Override

```
public void addPayment(Payment payment) {  
    String query = "INSERT INTO payments (payment_id, student_id,  
amount, payment_date) VALUES (?, ?, ?, ?)";  
  
    try (Connection con = DBConnUtil.getDbConnection();  
        PreparedStatement ps = con.prepareStatement(query)) {
```



```
ps.setInt(1, payment.getPaymentId());
ps.setInt(2, payment.getStudent().getStudentId());
ps.setDouble(3, payment.getAmount());
ps.setDate(4, new
java.sql.Date(payment.getPaymentDate().getTime()));
```

```
ps.executeUpdate();
} catch (SQLException e) {
    e.printStackTrace();
}
}
```

@Override

```
public List<Enrollment> getEnrollmentsForStudent(int studentId) {
    List<Enrollment> enrollments = new ArrayList<>();
    String query = "SELECT * FROM enrollments WHERE student_id =
?";
```

```
try (Connection con = DBConnUtil.getDbConnection();
    PreparedStatement ps = con.prepareStatement(query)) {
```

```
    ps.setInt(1, studentId);
    ResultSet rs = ps.executeQuery();
```

```
    while (rs.next()) {
        String enrollmentId = rs.getString("enrollment_id");
```

```

        Date enrollmentDate = rs.getDate("enrollment_date");
        Course course = getCourseById(rs.getString("course_id"));
        Student student = getStudentById(studentId);
        enrollments.add(new Enrollment(enrollmentId, student, course,
enrollmentDate));
    }
} catch (SQLException e) {
    e.printStackTrace();
}
return enrollments;
}

```

@Override

```

public List<Course> getAllCourses() {
    List<Course> courses = new ArrayList<>();
    String query = "SELECT * FROM courses";

    try (Connection con = DBConnUtil.getDbConnection();
        PreparedStatement ps = con.prepareStatement(query);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            String courseId = rs.getString("course_id");
            String courseName = rs.getString("course_name");
            int credits = rs.getInt("credits");
            int teacher_id=rs.getInt("teacher_id");

```

```

        String courseCode = rs.getString("course_code");

        courses.add(new Course(courseId, courseName,
credits,teacher_id, courseCode));
    }
} catch (SQLException e) {
    e.printStackTrace();
}
return courses;
}

```

@Override

```

public List<Student> getAllStudents() {
    List<Student> students = new ArrayList<>();
    String query = "SELECT * FROM students";

    try (Connection con = DBConnUtil.getDbConnection();
        PreparedStatement ps = con.prepareStatement(query);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            int studentId = rs.getInt("student_id");
            String firstName = rs.getString("first_name");
            String lastName = rs.getString("last_name");
            String dateOfBirth = rs.getString("date_of_birth");
            String email = rs.getString("email");

```

```

        String phoneNumber = rs.getString("phone_number");

        students.add(new Student(studentId, firstName, lastName,
dateOfBirth, email, phoneNumber));
    }
} catch (SQLException e) {
    e.printStackTrace();
}
return students;
}

```

@Override

```

public List<Teacher> getAllTeachers() {
    List<Teacher> teachers = new ArrayList<>();
    String query = "SELECT * FROM teacher";

    try (Connection con = DBConnUtil.getDbConnection();
        PreparedStatement ps = con.prepareStatement(query);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            int teacherId = rs.getInt("teacher_id");
            String firstName = rs.getString("first_name");
            String lastName = rs.getString("last_name");
            String email = rs.getString("email");

```

```

        teachers.add(new Teacher(teacherId, firstName, lastName,
email));
    }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return teachers;
}

```

`@Override`

```

public List<Payment> getAllPayments() {
    List<Payment> payments = new ArrayList<>();
    String query = "SELECT * FROM payments";

    try (Connection con = DBConnUtil.getDbConnection();
        PreparedStatement ps = con.prepareStatement(query);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            int paymentId = rs.getInt("payment_id");
            int studentId = rs.getInt("student_id");
            double amount = rs.getDouble("amount");
            Date paymentDate = rs.getDate("payment_date");

            Student student = getStudentById(studentId);
            payments.add(new Payment(paymentId, student, amount,
paymentDate));
        }
    }
}

```

```

    }
} catch (SQLException e) {
    e.printStackTrace();
}
return payments;
}

```

```

private Course getCourseById(String courseId) {
    String query = "SELECT * FROM courses WHERE course_id = ?";
    Course course = null;

    try (Connection con = DBConnUtil.getDbConnection();
        PreparedStatement ps = con.prepareStatement(query)) {

        ps.setString(1, courseId);
        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            String courseName = rs.getString("course_name");
            int credits = rs.getInt("credits");
            int teacher_id=rs.getInt("teacher_id");
            String courseCode = rs.getString("course_code");
            course = new Course(courseId, courseName, credits,
teacher_id,courseCode);
        }
    } catch (SQLException e) {

```

```
        e.printStackTrace();
    }
    return course;
}
```

@Override

```
public void assignCourseToTeacher(Course course, Teacher teacher) {
    String query = "UPDATE courses SET teacher_id = ? WHERE
course_id = ?";
```

```
    try (Connection con = DBConnUtil.getDbConnection();
        PreparedStatement ps = con.prepareStatement(query)) {

        ps.setInt(1, teacher.getTeacherId());
        ps.setString(2, course.getCourseId());

        ps.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

@Override

```
public Student getStudentById(int studentId) {
    String query = "SELECT * FROM students WHERE student_id = ?";
```

```

Student student = null;

try (Connection con = DBConnUtil.getDbConnection();
    PreparedStatement ps = con.prepareStatement(query)) {

    ps.setInt(1, studentId);

    ResultSet rs = ps.executeQuery();

    if (rs.next()) {
        String firstName = rs.getString("first_name");
        String lastName = rs.getString("last_name");
        String dateOfBirth = rs.getString("date_of_birth");
        String email = rs.getString("email");
        String phoneNumber = rs.getString("phone_number");
        student = new Student(studentId, firstName, lastName,
dateOfBirth, email, phoneNumber);
    }
} catch (SQLException e) {
    e.printStackTrace();
}

return student;
}

```

@Override

```

public Teacher getTeacherById(int teacherId) {

    String query = "SELECT * FROM teacher WHERE teacher_id = ?";

```



```

Teacher teacher = null;

try (Connection con = DBConnUtil.getDbConnection();
    PreparedStatement ps = con.prepareStatement(query)) {

    ps.setInt(1, teacherId);
    ResultSet rs = ps.executeQuery();

    if (rs.next()) {
        String firstName = rs.getString("first_name");
        String lastName = rs.getString("last_name");
        String email = rs.getString("email");

        teacher = new Teacher(teacherId, firstName, lastName, email);
    }
} catch (SQLException e) {
    e.printStackTrace();
}

return teacher;
}

```

@Override

```

public Course getCourseById(int courseId) {
    String query = "SELECT * FROM courses WHERE course_id = ?";
    Course course = null;

```

```

try (Connection con = DBConnUtil.getDbConnection();
    PreparedStatement ps = con.prepareStatement(query)) {

    ps.setInt(1, courseId);
    ResultSet rs = ps.executeQuery();

    if (rs.next()) {
        String courseIdStr = rs.getString("course_id");
        String courseName = rs.getString("course_name");
        int credits = rs.getInt("credits");
        int teacher_id=rs.getInt("teacher_id");
        String courseCode = rs.getString("course_code");

        course = new Course(courseIdStr, courseName, credits,
courseCode);
    }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return course;
}

@Override
public List<Enrollment> generateEnrollmentReport(String courseName
){
    List<Enrollment> enrollments = new ArrayList<>();

```

```

String courseQuery = "SELECT * FROM courses WHERE
course_name = ?";

String courseId = null;

try (Connection con = DBConnUtil.getDbConnection();
    PreparedStatement ps = con.prepareStatement(courseQuery)) {

    ps.setString(1, courseName);
    ResultSet rs = ps.executeQuery();

    if (rs.next()) {
        courseId = rs.getString("course_id");
    }

} catch (SQLException e) {
    e.printStackTrace();
}

if (courseId != null) {
    String enrollmentQuery = "SELECT * FROM enrollments
WHERE course_id = ?";

    try (Connection con = DBConnUtil.getDbConnection();
        PreparedStatement ps =
con.prepareStatement(enrollmentQuery)) {

```

```

ps.setString(1, courseId);
ResultSet rs = ps.executeQuery();

while (rs.next()) {
    String enrollmentId = rs.getString("enrollment_id");
    Date enrollmentDate = rs.getDate("enrollment_date");
    int studentId = rs.getInt("student_id");

    Student student = getStudentById(studentId);

    Course course = getCourseById(courseId);
    enrollments.add(new Enrollment(enrollmentId, student,
course, enrollmentDate));
}

} catch (SQLException e) {
    e.printStackTrace();
}
} else {
    System.out.println("Course with the name " + courseName + " not
found.");
}

return enrollments;
}

```

@Override

```
public List<Course> getCoursesForTeacher(int teacherId) {  
    List<Course> courses = new ArrayList<>();  
    String query = "SELECT * FROM courses WHERE teacher_id = ?";  
  
    try (Connection con = DBConnUtil.getDbConnection();  
        PreparedStatement ps = con.prepareStatement(query)) {  
  
        ps.setInt(1, teacherId);  
        ResultSet rs = ps.executeQuery();  
  
        while (rs.next()) {  
            String courseId = rs.getString("course_id");  
            String courseName = rs.getString("course_name");  
            int credits = rs.getInt("credits");  
            String courseCode = rs.getString("course_code");  
  
            courses.add(new Course(courseId, courseName, credits,  
teacherId, courseCode));  
        }  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    return courses;  
}
```

```
}
```

Exception Package

1. CourseNotFoundException Class

```
package exception;
```

```
public class CourseNotFoundException extends  
Exception {
```

```
    public CourseNotFoundException(String message) {  
        super(message);  
    }
```

```
}
```

2. DuplicateEnrollmentException Class

```
package exception;
```

```
public class DuplicateEnrollmentException extends  
Exception {
```

```
    public DuplicateEnrollmentException(String  
message) {  
        super(message);  
    }
```

```
}
```

3. InsufficientFundsException Class

```
package exception;

public class InsufficientFundsException extends
Exception {

    public InsufficientFundsException(String message) {
        super(message);
    }
}
```

4. InvalidCourseDataException Class

```
package exception;

public class InvalidCourseDataException extends
Exception {

    public InvalidCourseDataException(String message)
{
        super(message);
    }
}
```

5. InvalidEnrollmentDataException Class

```
package exception;

public class InvalidEnrollmentDataException extends
Exception {

    public InvalidEnrollmentDataException(String
message) {
```

```
        super(message);  
    }  
}
```

6. InvalidStudentDataException Class

```
package exception;  
  
public class InvalidStudentDataException extends  
Exception {  
    public InvalidStudentDataException(String message)  
    {  
        super(message);  
    }  
}
```

7. PaymentValidationException Class

```
package exception;  
  
public class PaymentValidationException extends  
Exception {  
    public PaymentValidationException(String message)  
    {  
        super(message);  
    }  
}
```

8. StudentNotFoundException Class


```
package exception;

public class StudentNotFoundException extends
Exception {

    public StudentNotFoundException(String message) {
        super(message);
    }
}
```

9. TeacherNotFoundException Class

```
package exception;

public class TeacherNotFoundException extends
Exception {

    public TeacherNotFoundException(String message) {
        super(message);
    }
}
```

10. InvalidTeacherDataException Class

```
package exception;

public class InvalidTeacherDataException extends
Exception {

    public InvalidTeacherDataException(String message)
{
        super(message);
    }
}
```

```

    }
}
Util Package
1.DBConnUtil Class
package util;
import java.io.IOException;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnUtil {

    private static final String fileName =
"src/db.properties";

    public static Connection getDbConnection() {
        Connection con = null;
        String connString = null;
        try {
            connString =
DBPropertyUtil.getConnectionString(fileName);
        } catch (IOException e) {

```

```
        System.out.println("Connection String Creation  
Failed");  
        e.printStackTrace();  
        throw new RuntimeException("Failed to create  
connection string", e);  
    }
```

```
    if (connString != null) {  
        try {  
            con =  
DriverManager.getConnection(connString);  
        } catch (SQLException e) {  
            System.out.println("Error While Establishing  
DBConnection...");  
            e.printStackTrace();  
            throw new RuntimeException("Failed to  
establish DB connection", e);  
        }  
    }  
    return con;  
}  
}
```

2.DBPropertyUtil Class

```
package util;
```

```
import java.io.FileInputStream;
```

```
import java.io.IOException;
```

```
import java.util.Properties;
```

```
public class DBPropertyUtil {
```

```
    public static String getConnectionString(String  
fileName) throws IOException {
```

```
        String connStr = null;
```

```
        Properties props = new Properties();
```

```
        try (FileInputStream fis = new  
FileInputStream(fileName)) {
```

```
            props.load(fis);
```

```
        }
```

```
        String url = props.getProperty("db.url");
```

```
        String username =  
props.getProperty("db.username");
```

```

        String password =
props.getProperty("db.password");

        if (url != null && username != null && password
!= null) {

            connStr = url + "?user=" + username +
"&password=" + password;

        } else {

            throw new IOException("Missing database
connection properties.");

        }

        return connStr;

    }
}

```

Main Package

1.MainModule Class

```

package main;

import dao.ISisdbImpl;
import entity.Course;
import entity.Enrollment;
import entity.Payment;
import entity.Student;
import entity.Teacher;

```

```
import java.sql.Date;
import java.util.List;
import java.util.Scanner;
```

```
public class MainModule {
```

```
    public static void main(String[] args) {
        ISisdbImpl sisdb = new ISisdbImpl();
        Scanner sc = new Scanner(System.in);
```

```
        while (true) {
            System.out.println("\nChoose a task to perform:");
            System.out.println("1 - Add Student, Course, Enrollment");
            System.out.println("2 - Add Teacher and Assign Course");
            System.out.println("3 - Record Payment for Student");
            System.out.println("4 - Generate Enrollment Report");
            System.out.println("5 - Exit");
            System.out.print("Enter your choice: ");
            int choice = sc.nextInt();
            sc.nextLine();
```

```
            switch (choice) {
```

```
            case 1:
```

```
                System.out.println("Enter Student ID:");
```

```
int studentId = sc.nextInt();
sc.nextLine();
System.out.println("Enter First Name:");
String fname = sc.nextLine();
System.out.println("Enter Last Name:");
String lname = sc.nextLine();
System.out.println("Enter DOB (yyyy-mm-dd):");
String dob = sc.nextLine();
System.out.println("Enter Email:");
String email = sc.nextLine();
System.out.println("Enter Phone:");
String phone = sc.nextLine();

Student student = new Student(studentId, fname, lname, dob,
email, phone);
sisdb.addStudent(student);

// Loop to add two courses
for (int i = 1; i <= 2; i++) {
    System.out.println("\nEnter Course ID " + i + ":");
    String cid = sc.nextLine();
    System.out.println("Enter Course Name " + i + ":");
    String cname = sc.nextLine();
    System.out.println("Enter Credits " + i + ":");
    int credits = sc.nextInt();
    sc.nextLine();
}
```

```

        System.out.println("Enter Teacher ID " + i + ":");
        int tid = sc.nextInt();
        sc.nextLine();
        System.out.println("Enter Course Code " + i + ":");
        String code = sc.nextLine();

        Course course = new Course(cid, cname, credits, tid, code);
        sisdb.addCourse(course);

        System.out.println("Enter Enrollment ID for course " + i + ":");
        String eid = sr.nextLine();
        System.out.println("Enter Enrollment Date for course " + i + "
(yyyy-mm-dd):");
        Date edate = Date.valueOf(sc.nextLine());

        Enrollment enrollment = new Enrollment(eid, student, course,
edate);
        sisdb.addEnrollment(enrollment);
    }

    System.out.println("Task 8 completed successfully...");
    break;

case 2:
    System.out.println("Enter Teacher ID:");
    int tid = sc.nextInt();

```



```
sc.nextLine();  
System.out.println("Enter First Name:");  
String tfname = sc.nextLine();  
System.out.println("Enter Last Name:");  
String tlname = sc.nextLine();  
System.out.println("Enter Email:");  
String temail = sc.nextLine();
```

```
Teacher teacher = new Teacher(tid, tfname, tlname, temail);  
sisdb.addTeacher(teacher);
```

```
System.out.println("Enter New Course ID:");  
String cid2 = sc.nextLine();  
System.out.println("Enter Course Name:");  
String cname2 = sc.nextLine();  
System.out.println("Enter Credits:");  
int credits2 = sc.nextInt();  
sc.nextLine();  
System.out.println("Enter Teacher ID:");  
int tid2 = sc.nextInt();  
sc.nextLine();  
System.out.println("Enter Course Code:");  
String code2 = sc.nextLine();
```

```
Course course2 = new Course(cid2, cname2, credits2, tid2,  
code2);
```

```
sisdb.addCourse(course2);  
sisdb.assignCourseToTeacher(course2, teacher);
```

```
System.out.println(teacher.getFirstName() + " assigned to " +  
course2.getCourseName() + " successfully.");  
System.out.println("Task 9 completed successfully...");  
break;
```

case 3:

```
System.out.println("Enter Student ID:");  
int pid = sc.nextInt();  
sc.nextLine();  
System.out.println("Enter First Name:");  
String pfname = sc.nextLine();  
System.out.println("Enter Last Name:");  
String pname = sc.nextLine();  
System.out.println("Enter DOB (yyyy-mm-dd):");  
String pdob = sc.nextLine();  
System.out.println("Enter Email:");  
String pemail = sc.nextLine();  
System.out.println("Enter Phone:");  
String pphone = sc.nextLine();
```

```
Student pstudent = new Student(pid, pfname, pname, pdob,  
pemail, pphone);  
sisdb.addStudent(pstudent);
```

```
System.out.println("Enter Payment ID:");  
int payId = sc.nextInt();  
System.out.println("Enter Amount:");  
double amount = sc.nextDouble();  
sc.nextLine();  
System.out.println("Enter Payment Date (yyyy-mm-dd):");  
Date payDate = Date.valueOf(sc.nextLine());
```

```
Payment payment = new Payment(payId, pstudent, amount,  
payDate);  
sisdb.addPayment(payment);
```

```
System.out.println("Payment of $" + amount + " recorded for "  
+ pfname + " " + pname + " successfully...");  
System.out.println("Task 10 completed successfully...");  
break;
```

case 4:

```
System.out.println("Enter Course Name for Enrollment  
Report:");  
String targetCourse = sc.nextLine();  
List<Enrollment> csEnrollments =  
sisdb.generateEnrollmentReport(targetCourse);
```

```
System.out.println("\nEnrollment Report for: " + targetCourse);  
for (Enrollment enrollment : csEnrollments) {  
    Student s = enrollment.getStudent();
```

```
        System.out.println("Student: " + s.getFirstName() + " " +
s.getLastName() +
        "--Enrollment Date: " +
enrollment.getEnrollmentDate());
    }

    System.out.println("Task 11 completed successfully...");
    break;
case 5:
    System.out.println("Exiting program..");
    System.exit(0);

default:
    System.out.println("Invalid choice. Please select a valid task
number.");
    }

    }
    }
}
```

OUTPUTS

```
Choose a task to perform:
1 - Add Student, Course, Enrollment
2 - Add Teacher and Assign Course
3 - Record Payment for Student
4 - Generate Enrollment Report
5 - Exit
Enter your choice: 1
Enter Student ID:
12
Enter First Name:
John
Enter Last Name:
Doe
Enter DOB (yyyy-mm-dd):
1995-08-15
Enter Email:
john.doe@example.com
Enter Phone:
1234567890
```

```
Enter Course ID 1:
C21
Enter Course Name 1:
Introduction to Programming
Enter Credits 1:
5
Enter Teacher ID 1:
2
Enter Course Code 1:
PY112
Enter Enrollment ID for course 1:
E12
Enter Enrollment Date for course 1 (yyyy-mm-dd):
2023-03-03

Enter Course ID 2:
C22
Enter Course Name 2:
Mathematics 101
Enter Credits 2:
4
Enter Teacher ID 2:
3
Enter Course Code 2:
MA134
Enter Enrollment ID for course 2:
E13
Enter Enrollment Date for course 2 (yyyy-mm-dd):
2023-03-03
Task 8 completed successfully...
```

```
Choose a task to perform:
1 - Add Student, Course, Enrollment
2 - Add Teacher and Assign Course
3 - Record Payment for Student
4 - Generate Enrollment Report
5 - Exit
Enter your choice: 2
Enter Teacher ID:
11
Enter First Name:
Sarah
Enter Last Name:
Smith
Enter Email:
sarah.smith@example.com
Enter New Course ID:
C01
Enter Course Name:
Advanced Database Management
Enter Credits:
3
Enter Teacher ID:
11
Enter Course Code:
CS302
Sarah assigned to Advanced Database Management successfully.
Task 9 completed successfully...
```

```
Choose a task to perform:
1 - Add Student, Course, Enrollment
2 - Add Teacher and Assign Course
3 - Record Payment for Student
4 - Generate Enrollment Report
5 - Exit
Enter your choice: 3
Enter Student ID:
101
Enter First Name:
Jane
Enter Last Name:
Johnson
Enter DOB (yyyy-mm-dd):
2000-05-25
Enter Email:
jane.johnson@gmail.com
Enter Phone:
6345782310
Enter Payment ID:
11
Enter Amount:
500.00
Enter Payment Date (yyyy-mm-dd):
2025-03-11
Payment of $500.0 recorded for Jane Johnson successfully...
Task 10 completed successfully...
```

Choose a task to perform:

- 1 - Add Student, Course, Enrollment
- 2 - Add Teacher and Assign Course
- 3 - Record Payment for Student
- 4 - Generate Enrollment Report
- 5 - Exit

Enter your choice: 4

Enter Course Name for Enrollment Report:

Computer Science 101

Enrollment Report for: Computer Science 101

Student: Jane Smith--Enrollment Date: 2025-03-16

Task 11 completed successfully...

Choose a task to perform:

- 1 - Add Student, Course, Enrollment
- 2 - Add Teacher and Assign Course
- 3 - Record Payment for Student
- 4 - Generate Enrollment Report
- 5 - Exit

Enter your choice: 5

Exiting program..